Fundamentals Before IO Operation

👉 Introduction to I/O in Java

Input/Output (I/O) in Java refers to the way a Java program exchanges data with external sources like files, databases, or other devices. For beginners, I/O operations are essential for making your programs interact with the outside world, allowing you to save and retrieve data permanently.

👉 Understanding Memory and Program Execution

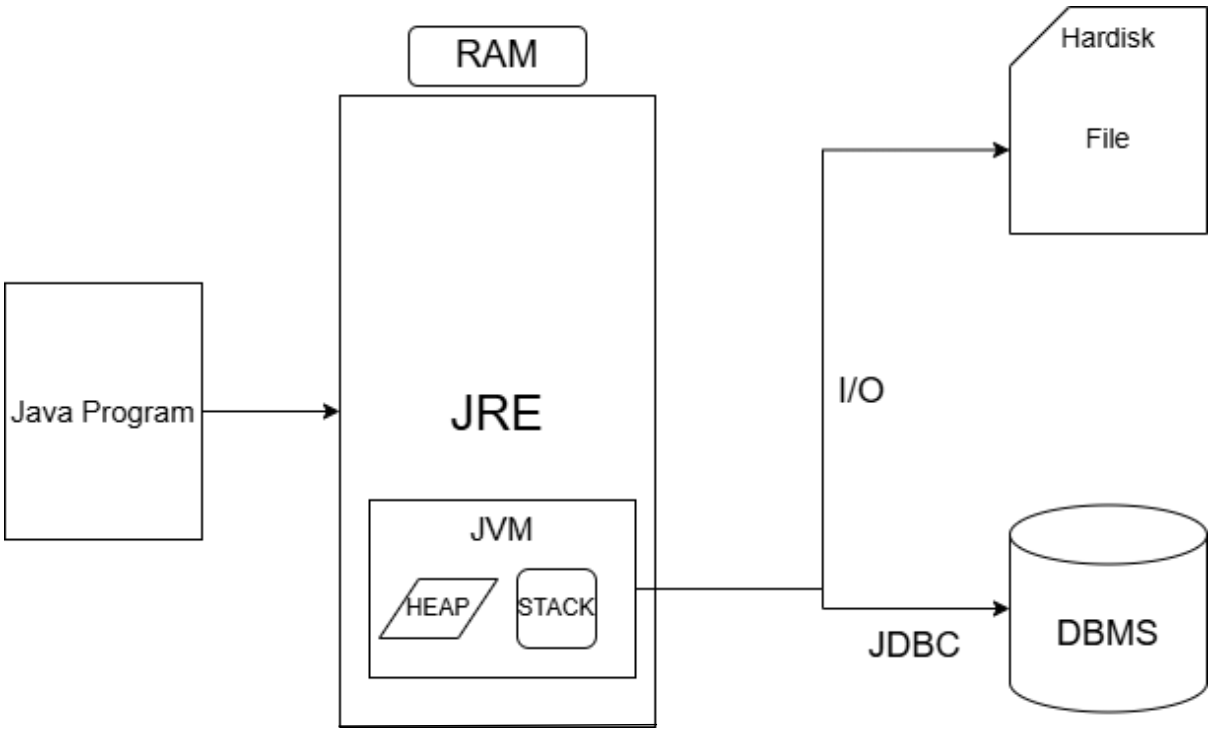
- Let's look at a simple Java program and understand what happens when we run it:

```java
class Student {

    private int id;
    private String name;
    private int age;

    // Parameterized constructor to initialize student data
    public Student(int id, String name, int age) {
        this.age = age;
        this.id = id;
        this.name = name;
    }

    // Overriding toString() to provide a meaningful string representation of the object
    @Override
    public String toString() {
        return "Student [id=" + id + ", name=" + name + ", age=" + age + "]";
    }
}


public class LaunchIO {
    public static void main(String[] args) {
        // Creating first Student object and printing its details
        Student s1 = new Student(1, "Rohan", 16);
        System.out.println(s1);  // Will call the overridden toString() method

        // Creating second Student object and printing its details
        Student s2 = new Student(2, "Rohit", 15);
        System.out.println(s2);  // Will call the overridden toString() method
    }
}
```

Output:

```
Student [id=1, name=Rohan, age=16]
Student [id=2, name=Rohit, age=15]
```

👉 What Happens When We Run This Program?



1. Program Loading to RAM

- When you run the Java program, it's loaded into RAM (Random Access Memory)
- RAM is a temporary storage location that provides fast access to data

2. Java Runtime Environment (JRE)

- JRE is created in RAM to execute your program
- JRE contains the Java Virtual Machine (JVM) and standard libraries

3. JVM Execution Process

- The JVM is responsible for executing your Java code
- While running your program, JVM uses special memory areas:
  - Heap: Stores objects (like our Student instances)
  - Stack: Manages method calls and local variables

4. Program Termination

- Once the program finishes execution, the JVM shuts down
- All memory allocated for the heap and stack is deallocated
- **Important**: All data in RAM is lost when the program ends!

5. The Temporary Nature of RAM

- In our example, the Student objects (s1 and s2) only exist in RAM while the program runs
- After execution, this data vanishes – we'd need to run the program again to recreate it

👉 The Need for Persistence

The problem with our current program is that all data exists only temporarily in RAM. What if we want to:

- Save student records permanently?
- Access the data later without rerunning the program?
- Share the data with other programs?

This is where we need **persistence** – the ability to store data permanently.

👉 Achieving Persistence in Java

There are two main ways to achieve persistence:

- **Using Hard Disk Storage (Files)**
  - Data can be saved to files on your hard disk
  - Files retain data even when your program isn't running
  - To connect your Java program to the hard disk, we use I/O operations
  - Java provides classes in the java.io package to read from and write to files

- **Using Database Management Systems (DBMS)**
  - Databases offer structured and efficient data storage
  - They provide additional features like searching, sorting, and securing data
  - To connect Java to a database, we use JDBC (Java Database Connectivity)

👉 Java I/O Package

The java.io package contains classes that allow your Java application to:

- Connect to files on your hard disk
- Store data permanently in files
- Read existing data from files back into your program

With I/O operations, you can modify our Student example to save the student information to a file, making it available even after the program ends.

👉 Summary

- Java programs run in RAM, which is temporary storage
- When a program ends, all its data in RAM is lost
- To keep data permanently, we need to use either:
  - Files on the hard disk (using Java I/O)
  - Databases (using JDBC)
- The java.io package provides classes for file operations
- I/O operations enable data persistence beyond program execution